出处说明：本文是论文《Distilling the Knowledge in a Neural Network》（Geoffrey Hinton, Oriol Vinyals, Jeff Dean, 2015）的中文译本，原文链接：https://arxiv.org/abs/1503.02531

蒸馏神经网络中的知识（Distilling the Knowledge in a Neural Network）

Geoffrey Hinton*† Google Inc.，Mountain View，geoffhinton@google.com **Oriol Vinyals**† Google Inc.，Mountain View，vinyals@google.com **Jeff Dean** Google Inc.，Mountain View，jeff@google.com

arXiv:1503.02531v1 [stat.ML]，2015 年 3 月 9 日

* 同时任职于多伦多大学（University of Toronto）和加拿大高等研究院（Canadian Institute for Advanced Research）。† 同等贡献。

译者注：原文没有插图，只有 5 张表格，下面已全部译出。公式用 LaTeX 写出。方括号里的数字（如 [3]）是参考文献编号，对应文末的参考文献列表。「译者注」都是为了帮助初学者理解而加的，原文没有。

摘要（Abstract）

要提升几乎任何机器学习算法的性能，有一个非常简单的办法：在同一份数据上训练许多不同的模型，然后对它们的预测取平均 [3]。可惜的是，用一整个模型集成（ensemble）来做预测非常笨重，计算开销也可能太大，难以部署给大量用户使用，单个模型是大型神经网络时尤其如此。Caruana 及其合作者 [1] 已经证明，可以把一个集成中的知识压缩到单个模型中，而单个模型要容易部署得多。我们用一种不同的压缩技术，把这一思路进一步发展。我们在 MNIST 上得到了一些出人意料的结果，并且证明：把一个模型集成中的知识蒸馏（distilling）到单个模型中，可以显著改进一个被大量使用的商业系统的声学模型（acoustic model）。我们还提出了一种新型集成，它由一个或多个完整模型（full model）以及许多专家模型（specialist model）组成，后者学习区分完整模型容易混淆的细粒度类别。与专家混合模型（mixture of experts）不同，这些专家模型可以快速地并行训练。

---

1 引言（Introduction）

许多昆虫都有一种幼虫形态，专门为从环境中汲取能量和养分而优化；还有一种完全不同的成虫形态，专门为迁移和繁殖这些截然不同的需求而优化。在大规模机器学习中，尽管训练阶段和部署阶段的需求差别很大，我们通常却在两个阶段使用非常相似的模型。对于语音识别和物体识别这类任务，训练必须从非常庞大、高度冗余的数据集中提取结构，但训练不需要实时运行，并且可以使用海量计算。而部署给大量用户使用时，对延迟（latency）和计算资源的要求则严格得多。昆虫的类比提示我们：如果训练非常笨重的模型能让我们更容易从数据中提取结构，那就应该愿意去训练这样的模型。这个笨重模型（cumbersome model）可以是多个分别训练的模型组成的集成，也可以是使用非常强的正则化项（regularizer），例如 dropout [9]，训练出来的单个超大模型。笨重模型训练好之后，我们可以使用另一种训练方式，我们称之为「蒸馏」（distillation），把笨重模型中的知识迁移到一个更适合部署的小模型中。Rich Caruana 及其合作者 [1] 已经率先尝试过这一策略的某种版本。他们在那篇重要的论文中令人信服地证明，一个大型模型集成所获得的知识可以迁移到单个小模型中。

有一个观念上的障碍，可能妨碍了人们对这一很有前景的方法做更多研究：我们往往把训练好的模型中的知识等同于学到的参数值，这让人很难想象，怎样才能在改变模型形式的同时保留同样的知识。对知识的一种更抽象的看法，可以让它摆脱任何具体的实现形式：知识就是一个学到的、从输入向量到输出向量的映射。对于学习区分大量类别的笨重模型，通常的训练目标是最大化正确答案的平均对数概率（log probability）。但学习带来一个副作用：训练好的模型会给所有错误答案也分配概率，即使这些概率都很小，其中一些仍比另一些大得多。错误答案之间的相对概率，能告诉我们很多关于笨重模型如何泛化（generalize）的信息。例如，一张宝马（BMW）汽车的图片被误认为垃圾车的可能性也许很小，但这种错误发生的概率仍然比把它误认为胡萝卜高出许多倍。

人们普遍认为，训练所用的目标函数（objective function）应该尽可能贴近使用者的真实目标。尽管如此，模型通常被训练为在训练数据上表现最优，而真正的目标却是在新数据上泛化得好。直接训练模型去泛化得好显然更好，但这需要知道正确的泛化方式，而这类信息通常无法获得。然而，当我们把知识从大模型蒸馏到小模型时，可以训练小模型以和大模型相同的方式去泛化。如果笨重模型泛化得好，比如因为它是一个由许多不同模型组成的大型集成的平均，那么一个被训练成以同样方式泛化的小模型，在测试数据上的表现通常会远好于另一个用常规方式、在训练该集成所用的同一训练集上训练出来的小模型。

要把笨重模型的泛化能力迁移给小模型，一个显而易见的办法，是把笨重模型产生的类别概率用作「软目标」（soft targets，也常译作「软标签」）来训练小模型。在这个迁移阶段，可以使用原来的训练集，也可以使用另外一个单独的「迁移集」（transfer set）。当笨重模型是由许多较简单模型组成的大型集成时，可以用这些模型各自预测分布的算术平均或几何平均作为软目标。当软目标的熵（entropy）很高时，与硬目标（hard targets）相比，每个训练样本能提供多得多的信息，不同训练样本之间梯度的方差也小得多。因此，小模型往往可以用比原笨重模型少得多的数据训练，并且可以使用高得多的学习率（learning rate）。

> 译者注：「硬目标」就是普通的独热（one-hot）标签，只告诉模型「正确答案是哪一类」；「软目标」是一个完整的概率分布，还告诉模型「其他类各有多像」。

对于像 MNIST 这样的任务，笨重模型几乎总能以非常高的置信度给出正确答案，关于所学函数的大量信息就存在于软目标里那些极小概率之间的比值中。例如，某一种写法的「2」被认为是「3」的概率可能是 10^{-6}，是「7」的概率是 10^{-9}；而另一种写法的「2」可能正好相反。这是很有价值的信息，它在数据上定义了一种丰富的相似性结构（即它说明了哪些 2 看起来像 3，哪些看起来像 7）。但由于这些概率太接近 0，它们在迁移阶段对交叉熵（cross-entropy）代价函数的影响非常小。Caruana 及其合作者绕开这个问题的办法是：不用 softmax 输出的概率，而是用 logits（即最后一层 softmax 的输入）作为训练小模型的目标，并最小化笨重模型产生的 logits 与小模型产生的 logits 之间的平方差。我们提出的更通用的解决方案称为「蒸馏」：提高最后一层 softmax 的温度（temperature），直到笨重模型产生一组足够「软」的目标。然后在训练小模型去拟合这些软目标时，也使用同样的高温。我们后面会说明，匹配笨重模型的 logits 实际上是蒸馏的一个特例。

用于训练小模型的迁移集，可以完全由无标签数据（unlabeled data）组成 [1]，也可以直接使用原始训练集。我们发现使用原始训练集效果很好，尤其是在目标函数中加入一个小的项，鼓励小模型在拟合笨重模型提供的软目标的同时，也去预测真实目标。通常小模型无法完全拟合软目标，而让误差偏向正确答案的方向，结果证明是有帮助的。

---

2 蒸馏（Distillation）

神经网络通常用一个「softmax」输出层来产生类别概率。对每个类别，它把计算出的 logit z_i 与其他 logits 相比较，从而转换成概率 q_i：

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{1}$$

其中 T 是温度，通常设为 1。T 取更高的值时，会在各类别上产生更软（更平缓）的概率分布。

> 译者注：举个例子，若 logits 为 (5, 2, 0)，$T=1$ 时概率约为 (0.95, 0.05, 0.01)，几乎是独热的；$T=5$ 时约为 (0.52, 0.29, 0.19)，各类之间的相对关系就清楚地暴露出来了。（数字为译者计算的示意值，不是论文数据。）

在最简单的蒸馏形式中，知识是这样迁移到蒸馏模型（distilled model）的：在迁移集上训练它，迁移集中每个样本的目标是一个软目标分布，这个分布由笨重模型在 softmax 中使用高温得到。训练蒸馏模型时使用同样的高温，但训练完成后，蒸馏模型使用的温度是 1。

当迁移集的全部或部分样本有正确标签时，再训练蒸馏模型去产生正确标签，可以显著改进这个方法。一种做法是用正确标签去修正软目标，但我们发现，更好的做法是直接对两个不同的目标函数做加权平均。第一个目标函数是与软目标之间的交叉熵，计算这个交叉熵时，蒸馏模型的 softmax 使用的高温与笨重模型生成软目标时所用的高温相同。第二个目标函数是与正确标签之间的交叉熵，计算时使用蒸馏模型 softmax 中完全相同的 logits，但温度为 1。我们发现，通常给第二个目标函数一个相当低的权重，能得到最好的结果。由于软目标产生的梯度大小按 $1/T^2$ 缩放，同时使用硬目标和软目标时，把软目标的梯度乘以 T^2 很重要。这能保证在调节超参数（原文作 meta-parameters）的实验中改变蒸馏温度时，硬目标和软目标的相对贡献大致保持不变。

> 译者注：写成公式，常见的蒸馏损失为 $L = \alpha\, T^2 \cdot \mathrm{CE}\big(p^{(T)}, q^{(T)}\big) + (1-\alpha)\cdot \mathrm{CE}\big(y, q^{(1)}\big)$，其中 $p^{(T)}$、$q^{(T)}$ 分别是老师和学生在温度 T 下的 softmax 输出，y 是真实标签。这一写法是对上文的概括，原文没有给出这个公式。

2.1 匹配 logits 是蒸馏的一个特例（Matching logits is a special case of distillation）

迁移集中的每个样本，都会对蒸馏模型的每个 logit z_i 贡献一个交叉熵梯度 dC/dz_i。如果笨重模型的 logits 为 v_i，它们产生的软目标概率为 p_i，迁移训练在温度 T 下进行，那么这个梯度为：

$$\frac{\partial C}{\partial z_i} = \frac{1}{T}(q_i - p_i) = \frac{1}{T}\left(\frac{e^{z_i/T}}{\sum_j e^{z_j/T}} - \frac{e^{v_i/T}}{\sum_j e^{v_j/T}}\right) \tag{2}$$

如果温度相对于 logits 的大小足够高，我们可以近似为：

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{T}\left(\frac{1 + z_i/T}{N + \sum_j z_j/T} - \frac{1 + v_i/T}{N + \sum_j v_j/T}\right) \tag{3}$$

> 译者注：这里用到了 x 很小时的近似 $e^x \approx 1 + x$；N 是类别总数。

如果我们再假设对每个迁移样本，logits 都已经分别做了零均值化，即 $\sum_j z_j = \sum_j v_j = 0$，那么式 (3) 可简化为：

$$\frac{\partial C}{\partial z_i} \approx \frac{1}{NT^2}(z_i - v_i) \tag{4}$$

因此，在高温极限下，只要每个迁移样本的 logits 分别做了零均值化，蒸馏就等价于最小化 $\frac{1}{2}(z_i - v_i)^2$。在较低的温度下，蒸馏对匹配那些远低于平均值（负得很多）的 logits 关注得少得多。这可能是有利的，因为训练笨重模型时所用的代价函数对这些 logits 几乎完全没有约束，它们可能噪声很大。另一方面，这些非常负的 logits 也可能携带关于笨重模型所获知识的有用信息。哪种效应占主导，是一个需要通过实验回答的问题。我们会说明，当蒸馏模型小到远远无法容纳笨重模型的全部知识时，中等温度效果最好，这强烈表明忽略那些很大的负 logits 可能是有益的。

---

3 MNIST 上的初步实验（Preliminary experiments on MNIST）

为了考察蒸馏的效果，我们在全部 60,000 个训练样本上训练了单个大型神经网络，它有两个隐藏层，每层 1200 个修正线性（rectified linear，即 ReLU）隐藏单元。该网络按照 [5] 中的描述，使用 dropout 和权重约束（weight constraints）进行了强正则化。dropout 可以看作一种训练指数级规模、共享权重的模型集成的方法。此外，输入图像在任意方向上最多抖动（jitter，即随机平移）两个像素。这个网络取得了 67 个测试错误；而一个较小的网络，有两个隐藏层、每层 800 个 ReLU 隐藏单元、不加任何正则化，则有 146 个错误。但如果对这个较小网络只做一种正则化，即额外加上一项任务：在温度 20 下拟合大网络产生的软目标，那么它的测试错误降到了 74 个。这说明软目标能把大量知识迁移给蒸馏模型，其中包括从平移过的训练数据中学到的关于如何泛化的知识，尽管迁移集本身并不包含任何平移。

当蒸馏网络两个隐藏层各有 300 个或更多单元时，所有高于 8 的温度给出的结果都相当接近。但当每层单元数大幅削减到 30 个时，2.5 到 4 范围内的温度比更高或更低的温度效果明显更好。

接着，我们尝试从迁移集中去掉所有数字 3 的样本。所以从蒸馏模型的角度看，3 是一个从未见过的「传说中的数字」。尽管如此，蒸馏模型只犯了 206 个测试错误，其中 133 个出现在测试集的 1010 个 3 上。大部分错误是因为模型为类别 3 学到的偏置（bias）太低了。如果把这个偏置增加 3.5（这个值使测试集上的整体表现最优），蒸馏模型只犯 109 个错误，其中 14 个在 3 上。因此，只要偏置合适，蒸馏模型在训练中从未见过 3 的情况下，仍能正确识别测试集中 98.6% 的 3。如果迁移集只包含训练集中的 7 和 8，蒸馏模型的测试错误率为 47.3%；但把 7 和 8 的偏置减少 7.6 以优化测试表现后，错误率降到 13.2%。

---

4 语音识别实验（Experiments on speech recognition）

本节我们研究对自动语音识别（Automatic Speech Recognition，ASR）中使用的深度神经网络（Deep Neural Network，DNN）声学模型做集成的效果。我们表明，本文提出的蒸馏策略达到了预期效果：把一个模型集成蒸馏成单个模型，这个模型的效果明显好于直接在同样训练数据上学习的同等规模模型。

当前最先进的 ASR 系统使用 DNN，把从波形中提取的特征的一个（短）时间上下文，映射为隐马尔可夫模型（Hidden Markov Model，HMM）离散状态上的概率分布 [4]。更具体地说，DNN 在每个时刻给出三音素（tri-phone）状态聚类上的概率分布，然后解码器（decoder）在 HMM 状态中寻找一条路径，在「使用高概率状态」和「产生在语言模型（language model）下概率较高的转写结果」之间取得最佳折中。

虽然可以（并且也希望）通过对所有可能路径做边缘化（marginalizing），以把解码器（进而语言模型）考虑在内的方式来训练 DNN，但常见做法是训练 DNN 做逐帧分类：（局部地）最小化网络预测与标签之间的交叉熵，这些标签来自每个观测与真实状态序列的强制对齐（forced alignment）：

$$\theta = \arg\max_{\theta'} P(h_t \mid s_t; \theta')$$

其中 θ 是声学模型 P 的参数。该模型把时刻 t 的声学观测 s_t 映射为「正确」HMM 状态 h_t 的概率 $P(h_t \mid s_t; \theta')$，而 h_t 由与正确词序列的强制对齐确定。模型使用分布式随机梯度下降（stochastic gradient descent）方法训练。

我们使用的架构有 8 个隐藏层，每层包含 2560 个 ReLU 单元，最后是一个有 14,000 个标签（HMM 目标 h_t）的 softmax 层。输入是 26 帧、每帧 40 个梅尔刻度滤波器组（Mel-scaled filterbank）系数，帧移为 10 毫秒，我们预测第 21 帧的 HMM 状态。参数总量约为 8500 万（85M）。这是 Android 语音搜索所用声学模型的一个稍旧的版本，应当被视为一个非常强的基线（baseline）。训练这个 DNN 声学模型时，我们使用了约 2000 小时的英语语音数据，产生约 7 亿（700M）个训练样本。该系统在我们的开发集（development set）上达到 58.9% 的帧准确率（frame accuracy）和 10.9% 的词错误率（Word Error Rate，WER）。

表 1： 帧分类准确率和 WER。结果表明，蒸馏得到的单个模型的表现与用来生成软目标的 10 个模型的平均预测大致相当。

系统	测试帧准确率	WER
基线（Baseline）	58.9%	10.9%
10 模型集成（10xEnsemble）	61.1%	10.7%
蒸馏得到的单个模型（Distilled Single model）	60.8%	10.7%

4.1 结果（Results）

我们训练了 10 个独立的模型来预测 $P(h_t \mid s_t; \theta)$，使用与基线完全相同的架构和训练流程。这些模型用不同的初始参数值随机初始化。我们发现，这样训练出的模型已经有足够的多样性，使集成的平均预测明显优于单个模型。我们也尝试过让每个模型看到不同的数据子集来增加多样性，但发现这对结果没有显著改变，因此选择了更简单的做法。在蒸馏中，我们尝试了温度 [1, **2**, 5, 10]，并给硬目标交叉熵设置 0.5 的相对权重，其中粗体表示表 1 所用的最佳取值。

> 译者注：pdftotext 提取的纯文本会丢失粗体。译者另用 PyMuPDF 检查 PDF 字体后确认，原文中加粗的是 2（字体为 CMBX10），即表 1 使用的蒸馏温度为 $T = 2$。

表 1 表明，我们的蒸馏方法确实能从训练集中提取比单纯用硬标签训练单个模型更多的有用信息。使用 10 个模型的集成在帧分类准确率上取得的提升，有超过 80% 被迁移到了蒸馏模型中，这与我们在 MNIST 初步实验中观察到的提升相似。由于目标函数不匹配，集成在最终目标 WER 上（在一个 2.3 万词的测试集上）带来的提升较小，但同样地，集成在 WER 上取得的提升也被迁移到了蒸馏模型中。

我们最近注意到一项相关工作 [8]：通过匹配一个已训练好的大模型的类别概率来学习小型声学模型。不过，他们在温度为 1 的条件下、使用一个大型无标签数据集做蒸馏；当大模型和小模型都用硬标签训练时，二者错误率之间存在一个差距，而他们最好的蒸馏模型只把小模型的错误率降低了这个差距的 28%。

---

5 在超大数据集上训练专家模型集成（Training ensembles of specialists on very big datasets）

训练模型集成是一种利用并行计算的非常简单的方法；而通常对集成的反对意见，即集成在测试时需要太多计算，可以用蒸馏来解决。然而，对集成还有另一个重要的反对意见：如果单个模型是大型神经网络，而数据集又

非常大，那么即使训练很容易并行化，训练时所需的计算量也过于庞大。

本节我们给出这样一个数据集的例子，并说明如何通过学习专家模型来减少学习一个集成所需的总计算量，其中每个专家模型专注于类别中一个不同的、容易混淆的子集。专注于细粒度区分的专家模型，主要问题是非常容易过拟合（overfit），我们会说明如何用软目标来防止这种过拟合。

5.1 JFT 数据集（The JFT dataset）

JFT 是 Google 内部的一个数据集, 包含 1 亿张带标签的图像, 共 15,000 个标签。在我们做这项工作时, Google 用于 JFT 的基线模型是一个深度卷积神经网络（deep convolutional neural network）[7]，它在大量 CPU 核心上使用异步随机梯度下降训练了大约六个月。这次训练用了两种并行方式 [2]。第一种：神经网络有许多副本（replica），运行在不同的核心组上，处理训练集中不同的小批量（mini-batch）。每个副本计算当前小批量上的平均梯度，并把梯度发送给一个分片的参数服务器（sharded parameter server），参数服务器再把参数的新值发回。这些新值反映了参数服务器自上次向该副本发送参数以来收到的所有梯度。第二种：每个副本通过把不同的神经元子集放到不同核心上，分布在多个核心上运行。集成训练是可以套在这两种方式外面的第三种并行方式，但前提是有多得多的核心可用。花好几年等待一个模型集成训练完成是不可行的，所以我们需要一种快得多的方法来改进基线模型。

表 2：由我们的协方差矩阵聚类算法得到的聚类中的示例类别

聚类	示例类别
JFT 1	茶会（Tea party）；复活节（Easter）；新娘送礼会（Bridal shower）；迎婴派对（Baby shower）；复活节兔子（Easter Bunny）；……
JFT 2	桥（Bridge）；斜拉桥（Cable-stayed bridge）；悬索桥（Suspension bridge）；高架桥（Viaduct）；烟囱（Chimney）；……
JFT 3	丰田卡罗拉 E100（Toyota Corolla E100）；欧宝 Signum（Opel Signum）；欧宝雅特（Opel Astra）；马自达 Familia（Mazda Familia）；……

5.2 专家模型（Specialist Models）

当类别数量非常大时，让笨重模型成为这样一个集成是合理的：它包含一个在全部数据上训练的通才模型（generalist model），以及许多「专家」模型，每个专家模型所用的训练数据中，某个非常容易混淆的类别子集（例如不同种类的蘑菇）的样本被大量富集。这类专家模型的 softmax 可以做得小得多：把它不关心的所有类别合并成一个「垃圾桶」类（dustbin class）。

为了减少过拟合，并共享学习低层特征检测器的工作，每个专家模型都用通才模型的权重初始化。然后训练专家模型，对这些权重做轻微修改：训练样本一半来自它的专属子集，另一半从训练集其余部分中随机抽取。训练结束后，为了校正有偏的训练集，可以把垃圾桶类的 logit 加上专家类别被过采样（oversampled）比例的对数。

5.3 为专家分配类别（Assigning classes to specialists）

为了给专家得出物体类别的分组，我们决定关注完整网络经常混淆的类别。虽然我们本可以计算混淆矩阵（confusion matrix）并用它来找这样的聚类，但我们选择了一种更简单的方法，它构建聚类时不需要真实标签。

具体来说，我们对通才模型预测结果的协方差矩阵（covariance matrix）应用聚类算法，这样，经常被一起预测的一组类别 S^m 就会被用作某个专家模型 m 的目标。我们对协方差矩阵的各列应用了在线版本的 K-means 算法，得到了合理的聚类（见表 2）。我们尝试过几种聚类算法，结果都差不多。

5.4 用专家集成做推断（Performing inference with ensembles of specialists）

在研究对专家模型做蒸馏会发生什么之前，我们想先看看包含专家的集成表现如何。除了专家模型之外，我们始终保留一个通才模型，这样既能处理没有对应专家的类别，也能决定使用哪些专家。给定输入图像 x，我们分两步做 top-1 分类：

第 1 步： 对每个测试样本，根据通才模型找出概率最高的 n 个类别，把这组类别记为 k。在我们的实验中，$n = 1$。

第 2 步： 然后取出所有满足以下条件的专家模型 m：其专属的易混淆类别子集 S^m 与 k 的交集非空。把这些专家称为激活专家集合 A_k（注意这个集合可能为空）。接着，我们求出所有类别上的完整概率分布 q，使下式最小：

$$KL(p^g, q) + \sum_{m \in A_k} KL(p^m, q) \tag{5}$$

其中 KL 表示 KL 散度（KL divergence），p^m 和 p^g 分别表示专家模型和通才完整模型的概率分布。分布 p^m 定义在 m 的所有专家类别外加一个垃圾桶类上，所以在计算它与完整分布 q 之间的 KL 散度时，我们把完整分布 q 分配给 m 垃圾桶中所有类别的概率加总起来。

式 (5) 一般没有闭式解（closed form solution）；不过，当所有模型都对每个类别给出单一概率时，解是算术平均或几何平均，取决于我们使用 $KL(p, q)$ 还是 $KL(q, p)$。我们把 q 参数化为 $q = \text{softmax}(z)$（取 $T = 1$），并用梯度下降针对式 (5) 优化 logits z。注意，这个优化必须对每张图像单独进行。

表 3： JFT 开发集上的分类准确率（top 1）

系统	条件测试准确率	测试准确率
基线（Baseline）	43.1%	25.0%
+ 61 个专家模型	45.9%	26.1%

表 4： 按「覆盖正确类别的专家模型数量」划分的 JFT 测试集 top-1 准确率提升

覆盖该类的专家数	测试样本数	top-1 正确数的变化	准确率相对变化
0	350037	0	0.0%
1	141993	+1421	+3.4%
2	67161	+1572	+7.4%
3	38801	+1124	+8.8%
4	26298	+835	+10.5%

覆盖该类的专家数	测试样本数	top-1 正确数的变化	准确率相对变化
5	16474	+561	+11.1%
6	10682	+362	+11.3%
7	7376	+232	+12.8%
8	4703	+182	+13.6%
9	4706	+208	+16.6%
10 个或更多	9082	+324	+14.1%

5.5 结果（Results）

从训练好的基线完整网络出发，专家模型训练得非常快（在 JFT 上只需几天，而不是好几周）。而且，所有专家都是完全独立训练的。表 3 给出了基线系统，以及基线系统与专家模型组合后的绝对测试准确率。使用 61 个专家模型后，整体测试准确率相对提升了 4.4%。我们还报告了条件测试准确率（conditional test accuracy），即只考虑属于专家类别的样本，并把预测限制在该类别子集内时的准确率。

在 JFT 专家实验中，我们训练了 61 个专家模型，每个有 300 个类别（外加垃圾桶类）。由于各个专家的类别集合并不互斥，经常有多个专家覆盖同一个图像类别。表 4 按覆盖该类别的专家数量分组，给出了 JFT 数据集上的测试样本数、使用专家后排在第 1 位正确的样本数的变化，以及 top-1 准确率的相对提升百分比。总体趋势是：覆盖某个类别的专家越多，准确率提升越大。这让我们深受鼓舞，因为独立训练的专家模型非常容易并行化。

---

6 软目标作为正则化项（Soft Targets as Regularizers）

关于用软目标代替硬目标，我们的一个主要论点是：软目标能携带大量有用信息，而这些信息不可能用单个硬目标来编码。本节我们用少得多的数据去拟合前面介绍的、有 8500 万参数的基线语音模型，以此证明这种效应非常显著。表 5 显示，只用 3% 的数据（约 2000 万个样本）时，用硬目标训练基线模型会导致严重过拟合（我们使用了早停（early stopping），因为准确率在达到 44.5% 后会急剧下降）；而用软目标训练同一个模型，能够恢复完整训练集中几乎全部的信息（只差约 2%）。更值得注意的是，我们不需要早停：使用软目标的系统直接「收敛」到了 57%。这说明，要把一个在全部数据上训练的模型所发现的规律传达给另一个模型，软目标是一种非常有效的方式。

表 5：软目标让新模型只用 3% 的训练集就能很好地泛化。软目标是通过在完整训练集上训练得到的。

系统与训练集	训练帧准确率	测试帧准确率
基线（100% 训练集）	63.4%	58.9%
基线（3% 训练集）	67.3%	44.5%
软目标（3% 训练集）	65.4%	57.0%

6.1 用软目标防止专家模型过拟合（Using soft targets to prevent specialists from overfitting）

我们在 JFT 数据集实验中使用的专家模型，把所有非专属类别合并成了单个垃圾桶类。如果允许专家模型拥有覆盖全部类别的完整 softmax，那么可能会有比早停好得多的防止过拟合的方法。专家模型的训练数据中，其专属类别被大量富集。这意味着它的有效训练集规模要小得多，并且很容易在专属类别上过拟合。这个问题不能通过把专家模型做得小很多来解决，因为那样我们会失去对所有非专属类别建模所带来的非常有益的迁移效果。

我们用 3% 语音数据做的实验强烈表明：如果用通才模型的权重初始化专家模型，那么在用硬目标训练它的同时，再对非专属类别用软目标训练它，就能让它几乎保留关于非专属类别的全部知识。这些软目标可以由通才模型提供。我们目前正在探索这一方法。

---

7 与专家混合模型的关系（Relationship to Mixtures of Experts）

使用在数据子集上训练的专家模型，与专家混合模型（mixtures of experts）[6] 有一些相似之处。专家混合模型用一个门控网络（gating network）计算把每个样本分配给每个专家的概率。在专家学习处理分配给它们的样本的同时，门控网络也在学习：根据各个专家在某个样本上的相对判别表现，决定把这个样本分配给哪些专家。用专家的判别表现来决定学到的分配方式，比简单地对输入向量聚类、再给每个聚类分配一个专家要好得多，但这会让训练难以并行化：第一，每个专家的加权训练集不断变化，而且变化方式依赖于所有其他专家；第二，门控网络需要比较不同专家在同一样本上的表现，才知道如何修正分配概率。这些困难意味着，专家混合模型很少被用在它可能最有用的场景中：数据集巨大、且包含明显不同子集的任务。

并行训练多个专家模型则容易得多。我们先训练一个通才模型，然后用混淆矩阵定义各专家所训练的子集。这些子集一旦确定，各专家就可以完全独立地训练。测试时，可以用通才模型的预测来决定哪些专家是相关的，只需运行这些专家即可。

> 译者注：第 5.3 节说聚类用的是通才模型预测的协方差矩阵，而本节说用混淆矩阵定义子集，原文前后表述略有出入，此处按原文照译。

---

8 讨论（Discussion）

我们已经证明，要把知识从一个集成或一个经过高度正则化的大模型迁移到一个更小的蒸馏模型中，蒸馏的效果非常好。在 MNIST 上，即使用来训练蒸馏模型的迁移集完全缺少一个或多个类别的样本，蒸馏的效果依然好得出奇。对于一个深度声学模型（Android 语音搜索所用模型的一个版本），我们证明了：通过训练深度神经网络集成所获得的几乎全部提升，都可以蒸馏到单个同等规模的神经网络中，而这个网络要容易部署得多。

对于真正巨大的神经网络，甚至训练一个完整的集成都可能不可行。但我们已经证明，通过学习大量专家网络，每个专家网络学习区分一个高度易混淆聚类中的各个类别，可以显著提升一个训练了非常长时间的超大单一网络的性能。我们尚未证明能把专家中的知识蒸馏回那个单一的大网络中。

致谢（Acknowledgments）

我们感谢 Yangqing Jia（贾扬清）在 ImageNet 模型训练上提供的帮助, 并感谢 Ilya Sutskever 和 Yoram Singer 进行的有益讨论。

参考文献（References）

译者注：参考文献保留原文，不做翻译，方便检索。

[1] C. Buciluǎ, R. Caruana, and A. Niculescu-Mizil. Model compression. In Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’06, pages 535–541, New York, NY, USA, 2006. ACM.

[2] J. Dean, G. S. Corrado, R. Monga, K. Chen, M. Devin, Q. V. Le, M. Z. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Y. Ng. Large scale distributed deep networks. In NIPS, 2012.

[3] T. G. Dietterich. Ensemble methods in machine learning. In Multiple classifier systems, pages 1–15. Springer, 2000.

[4] G. E. Hinton, L. Deng, D. Yu, G. E. Dahl, A. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, T. N Sainath, and B. Kingsbury. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. Signal Processing Magazine, IEEE, 29(6):82–97, 2012.

[5] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. arXiv preprint arXiv:1207.0580, 2012.

[6] R. A. Jacobs, M. I. Jordan, S. J. Nowlan, and G. E. Hinton. Adaptive mixtures of local experts. Neural computation, 3(1):79–87, 1991.

[7] A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In Advances in Neural Information Processing Systems, pages 1097–1105, 2012.

[8] J. Li, R. Zhao, J. Huang, and Y. Gong. Learning small-size dnn with output-distribution-based criteria. In Proceedings Interspeech 2014, pages 1910–1914, 2014.

[9] N. Srivastava, G.E. Hinton, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. The Journal of Machine Learning Research, 15(1):1929–1958, 2014.